

```

import { Circle } from "../Circle";
import { FreshDateMascot } from "@components/nakhlah/DateMascot";
import { useEffect, useMemo, useState } from "react";
import { motion } from "framer-motion";
import { Lock, FileText } from "lucide-react";

const PATH_CENTER = 50;
const PATH_AMPLITUDE = 25;
const PATH_FREQUENCY = 0.8;
const LESSON_ROW_HEIGHT = 112;
const MASCOT_VERTICAL_OFFSET = -150;
const MASCOT_SIDE_POSITIONS = {
  left: "22%",
  right: "78%",
};

export function ZigzagPath({ lessons, levels, mascots, isLoading = false }) {
  const [currentLevelId, setCurrentLevelId] = useState("");

  const currentLevel = levels.find((l) => l.id === currentLevelId);

  const groupedLessons = lessons.reduce((acc, lesson) => {
    const key = lesson.sectionId || lesson.level;
    if (!acc[key]) acc[key] = [];
    acc[key].push(lesson);
    return acc;
  }, {});

  const currentSectionLessons = currentLevel
    ? groupedLessons[currentLevel.id] || []
    : [];
  const currentTask =
    currentSectionLessons.find((lesson) => lesson.isCurrent) ||
    currentSectionLessons.find((lesson) => !lesson.isLocked) ||
    currentSectionLessons[0];

  const getPosition = (index) => {
    const x = PATH_CENTER + Math.sin(index * PATH_FREQUENCY) * PATH_AMPLITUDE;
    return { left: `${x}%`, transform: "translateX(-50%)" };
  };

  const lessonIndexById = useMemo(
    () => new Map(lessons.map((lesson, index) => [lesson.id, index])),
    [lessons],
  );

  const mascotPlacementsByAnchorId = useMemo(() => {
    if (!Array.isArray(lessons) || lessons.length < 4) return new Map();

    const moods = [
      "proud",
      "happy",
      "focused",
      "confident",
      "encouraging",
      "thinking",
      "celebrating",
      "surprised",
      "cool",
    ];

```

```

    "excited",
  ];
  const sizes = ["xxl", "xxl", "xxl", "xxl"];

  // Cycle through moods in order, repeating from the start after the last one
  const cycleMoods = (count) => {
    return Array.from({ length: count }, (_, i) => moods[i % moods.length]);
  };
  const halfWave = Math.PI / PATH_FREQUENCY;
  const firstTurningPoint = Math.PI / (2 * PATH_FREQUENCY);
  const slotCandidates = [];

  for (
    let turningPoint = firstTurningPoint, slotIndex = 0;
    turningPoint + halfWave <= lessons.length;
    turningPoint += halfWave, slotIndex += 1
  ) {
    const midpoint = turningPoint + halfWave / 2;
    const anchorIndex = Math.min(
      lessons.length - 1,
      Math.max(0, Math.floor(midpoint)),
    );

    slotCandidates.push({
      anchorLessonId: lessons[anchorIndex]?.id,
      midpoint,
      side: slotIndex % 2 === 0 ? "left" : "right",
      slotIndex,
    });
  }

  if (slotCandidates.length === 0) return new Map();

  const requestedMascots =
    Array.isArray(mascots) && mascots.length > 0
      ? mascots
      : cycleMoods(slotCandidates.length).map((mood, index) => ({
          mood,
          size: sizes[index % sizes.length],
        }));

  const availableSlots = [...slotCandidates];
  const placementsByAnchor = new Map();

  requestedMascots
    .slice(0, slotCandidates.length)
    .forEach((mascot, index) => {
      if (availableSlots.length === 0) return;

      const requestedIndex = mascot?.position
        ? lessonIndexById.get(mascot.position)
        : null;

      let slotChoiceIndex = Math.min(index, availableSlots.length - 1);

      if (Number.isInteger(requestedIndex)) {
        slotChoiceIndex = availableSlots.reduce(
          (bestIndex, slot, currentIndex) =>
            Math.abs(slot.midpoint - requestedIndex) <

```

```

    Math.abs(availableSlots[bestIndex].midpoint - requestedIndex)
    ? currentIndex
    : bestIndex,
    0,
  );
}

const [slot] = availableSlots.splice(slotChoiceIndex, 1);

if (!slot?.anchorLessonId) return;

const placement = {
  ...slot,
  mood: mascot?.mood || moods[slot.slotIndex % moods.length],
  size: mascot?.size || sizes[slot.slotIndex % sizes.length],
  message: mascot?.message,
};

const anchoredPlacements =
  placementsByAnchor.get(slot.anchorLessonId) || [];
anchoredPlacements.push(placement);
placementsByAnchor.set(slot.anchorLessonId, anchoredPlacements);
});

return placementsByAnchor;
}, [lessonIndexById, lessons, mascots]);

const getLevelColor = (level) => {
  const colors = [
    "from-green-400 to-green-600",
    "from-purple-400 to-purple-600",
    "from-orange-400 to-orange-600",
    "from-blue-400 to-blue-600",
    "from-red-400 to-red-600",
  ];
  return colors[(level - 1) % colors.length];
};

useEffect(() => {
  if (isLoading || !levels.length) return undefined;

  const observers = [];
  const levelElements = document.querySelectorAll("[data-level-id]");

  const observerCallback = (entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        const levelId = entry.target.getAttribute("data-level-id");
        const level = levels.find((l) => l.id.toString() === levelId);
        if (level) {
          setCurrentLevelId(level.id);
        }
      }
    });
  };

  const observer = new IntersectionObserver(observerCallback, {
    root: null,
    rootMargin: "-15% 0px -80% 0px",
  });

```

```

    threshold: 0,
  });

  levelElements.forEach((el) => {
    observer.observe(el);
    observers.push(observer);
  });

  if (levels && levels.length > 0 && !currentLevelId) {
    setCurrentLevelId(levels[0].id);
  }

  return () => observers.forEach((o) => o.disconnect());
}, [levels, isLoading, currentLevelId]);

useEffect(() => {
  if (isLoading || lessons.length === 0) return undefined;

  const lastInteractedId = sessionStorage.getItem("lastInteractedNodeId");
  const selectedId = sessionStorage.getItem("selectedNodeId");

  let targetEl = null;
  if (lastInteractedId) {
    targetEl = document.getElementById(`node-${lastInteractedId}`);
  }
  if (!targetEl && selectedId) {
    targetEl = document.getElementById(`node-${selectedId}`);
  }
  if (!targetEl) {
    const currentLesson = lessons.find((l) => l.isCurrent);
    if (currentLesson) {
      targetEl = document.getElementById(`node-${currentLesson.apid}`);
    }
  }
  if (!targetEl) {
    const firstUnlockedLesson = lessons.find((l) => !l.isLocked);
    if (firstUnlockedLesson) {
      targetEl = document.getElementById(`node-${firstUnlockedLesson.apid}`);
    }
  }

  if (targetEl) {
    targetEl.scrollIntoView({ behavior: "auto", block: "center" });
  }
}, [lessons, isLoading]);

return (
  <div className="relative lg:max-w-lg mx-auto">
    {/* Mobile mask: hides scrolled content behind the fixed stats + unit headers */}
    <div className="fixed top-0 left-0 right-0 h-[72px] z-[44] bg-background lg:hidden pointer-events-none" />

    {/* Fixed unit header on mobile to avoid sticky jitter while scrolling */}
    <div className="fixed top-[72px] left-0 right-0 z-[45] bg-background border-b border-border/60 py-2 lg:hidden">
      <div className="mx-auto w-full max-w-lg px-4">
        <div
          className={`flex items-center justify-between px-4 py-3 rounded-lg shadow-lg transition-all duration-500
ease-in-out bg-gradient-to-r ${getLevelColor(
            currentLevel?.colorIndex || 1,
          )} text-white`}

```

```

>
<div>
  <div className="text-sm text-white/90 mb-1 font-semibold uppercase tracking-wider">
    {currentLevel?.levelName ? `${currentLevel.levelName}, ` : ""}
    {currentLevel?.name || ""}
  </div>
  {currentTask?.title ? (
    <div className="text-2xl font-bold leading-tight">
      {currentTask.title}
    </div>
  ) : null}
</div>
<button className="text-white p-2 rounded-full">
  <FileText className="w-6 h-6" />
</button>
</div>
</div>
</div>

{/* Desktop mask to keep whitespace above sticky header clean */}
<div className="hidden lg:block sticky top-0 z-[44] h-6 bg-background" />

{/* Sticky unit header on desktop */}
<div className="hidden lg:block sticky top-6 z-[44] bg-background py-2 lg:py-0">
  <div
    className={`flex items-center justify-between px-4 py-3 rounded-lg shadow-lg transition-all duration-500
ease-in-out bg-gradient-to-r ${getLevelColor(
  currentLevel?.colorIndex || 1,
)} text-white`}
  >
    <div>
      <div className="text-sm text-white/90 mb-1 font-semibold uppercase tracking-wider">
        {currentLevel?.levelName ? `${currentLevel.levelName}, ` : ""}
        {currentLevel?.name || ""}
      </div>
      {currentTask?.title ? (
        <div className="text-2xl font-bold leading-tight">
          {currentTask.title}
        </div>
      ) : null}
    </div>
    <button className="text-white p-2 rounded-full">
      <FileText className="w-6 h-6" />
    </button>
  </div>
</div>

<div className="h-[88px] lg:hidden" />

{/* Lessons grouped by level */}
<div className="relative mt-6 lg:mt-10">
  {levels.map((level, levelIndex) => {
    const levelLessons = groupedLessons[level.id] || [];
    const isFirstLessonCurrent = levelLessons[0]?.isCurrent;
    const levelStartIndex = lessonIndexById.get(levelLessons[0]?.id) ?? 0;
    const levelMascots = levelLessons.flatMap(
      (lesson) => mascotPlacementsByAnchorId.get(lesson.id) || [],
    );
  })}

```

```

return (
  <div
    key={level.id}
    data-level-id={level.id}
    className="mb-12 relative"
  >
    {/ * Level barrier */}
    <div
      className={`relative h-1 flex items-center justify-center ${
        isFirstLessonCurrent ? "mb-16 mt-6" : "mb-6"
      }`}
    >
      {/ * <div className="absolute inset-0 flex items-center">
        <div className="w-full border-t-2 border-dashed border-border"></div>
      </div> */}
      <div className="relative bg-background px-4">
        <span
          className={`text-lg font-bold bg-gradient-to-r ${getLevelColor(
            level.colorIndex || levelIndex + 1,
          )} bg-clip-text text-transparent`}
        >
          {level.name}
        </span>
      </div>
    </div>

    {/ * Zigzag path for this level */}
    <div className="relative">
      {levelLessons.map((lesson, index) => {
        const globalIndex = lessonIndexByld.get(lesson.id);
        const position = getPosition(globalIndex ?? index);

        return (
          <div
            key={lesson.id}
            id={`node-${lesson.apild}`}
            className="relative h-28 w-full"
          >
            {/ * Lesson circle */}
            <div
              className="absolute"
              style={{
                left: position.left,
                top: "50%",
                transform: `${position.transform} translateY(-50%)`,
              }}
            >
              <Circle
                isCompleted={lesson.isCompleted}
                isCurrent={lesson.isCurrent}
                isLocked={lesson.isLocked}
                icon={lesson.icon}
                type={lesson.type}
                size="lg"
                nodeId={lesson.apild}
              />
            </div>

            {/ * "Speech / quotation" bubble - positioned directly above the node */}

```

```

{lesson.isCurrent && (
  <div
    aria-hidden
    className="absolute z-10"
    style={{
      left: position.left,
      top: "-40%",
      transform: "translateX(-50%)",
    }}
  >
    <div
      className="relative mx-auto bg-white text-sm font-semibold px-4 py-2 rounded-2xl shadow-md
border-accent w-max min-w-[100px]"
      style={{
        borderWidth: 4,
      }}
    >
      {/* The bubble text */}
      <div className="text-center font-bold text-accent tracking-wide uppercase">
        START!
      </div>

      {/* Beautiful curved SVG tail */}
      <svg
        className="absolute left-1/2 -translate-x-1/2 text-accent"
        style={{
          bottom: -14,
        }}
        width="24"
        height="14"
        viewBox="0 0 24 14"
        fill="none"
        xmlns="http://www.w3.org/2000/svg"
      >
        <rect
          x="-1"
          y="-1"
          width="26"
          height="5"
          fill="white"
        />
        <path
          d="M-1 2 C 8 2, 8 12, 12 12 C 16 12, 16 2, 25 2"
          stroke="currentColor"
          strokeWidth="4"
          strokeLinejoin="round"
          fill="white"
        />
      </svg>
    </div>
  </div>
)}
</div>
);
}}

{levelMascots.map((mascot) => (
  <div
    key={` ${level.id}-${mascot.slotIndex}-${mascot.mood || "helper"}`}

```

```

        className="absolute z-10 pointer-events-none"
        style={{
          left: MASCOT_SIDE_POSITIONS[mascot.side],
          top: `${
            (mascot.midpoint - levelStartIndex) *
            LESSON_ROW_HEIGHT +
            LESSON_ROW_HEIGHT / 2 +
            MASCOT_VERTICAL_OFFSET
          }px`,
          transform: "translate(-50%, -50%)",
        }}
      >
      <FreshDateMascot
        mood={mascot.mood || "happy"}
        size={mascot.size || "md"}
        message={mascot.message}
      />
    </div>
  )}
</div>
</div>
);
}}}

{/* Section unlocker placeholder */}
<div className="mt-8 flex justify-center">
  <div className="bg-card border-2 border-dashed border-border rounded-xl p-6 w-full max-w-md text-center">
    <div className="flex justify-center mb-3">
      <Lock className="w-6 h-6 text-muted-foreground" />
    </div>
    <h3 className="text-lg font-bold text-foreground mb-2">
      Next Section Locked
    </h3>
    <p className="text-sm text-muted-foreground mb-4">
      Complete the current section to unlock the next one.
    </p>
  </div>
</div>
</div>
</div>
);
}

```